Kubernetes Autoscaling and RBAC

Topics: Horizontal Pod Autoscaler (HPA), Vertical Pod Autoscaler (VPA), Cluster Autoscaler, Role-Based Access Control (RBAC), ServiceAccounts, and least privilege.

Part 1 — Kubernetes Autoscaling

1. Why Kubernetes Applications Need Autoscaling

Consider a Spring Boot application deployed with two replicas:

```
spec:
  replicas: 2
```

Under normal traffic, these two Pods may be enough. However, if traffic increases from **100 requests/second to 1,000 requests/second**, the application can start experiencing:

- Higher CPU usage
- More concurrent requests and longer queues
- Slower response times
- Timeouts and failed requests when capacity runs out

A regular Kubernetes Deployment maintains the replica count specified in its configuration. **It does not add more replicas simply because traffic has increased.** Autoscaling helps adjust capacity as demand changes.

Three ways to increase capacity

Scaling method	What changes	Example
Horizontal scaling	Number of application instances (Pods)	Increase Spring Boot Pods from 2 to 5
Vertical scaling	CPU/memory allocation for individual Pods	Increase a Pod's requested memory from 128Mi to 512Mi
Node/cluster scaling	Number of worker nodes	Add another worker node when existing nodes are full

Kubernetes has different autoscaling mechanisms for these different problems.

---

2. Horizontal Pod Autoscaler (HPA)

Horizontal Pod Autoscaler (HPA) automatically adjusts the number of replicas in a scalable workload, such as a Deployment, using observed metrics.

It can use metrics such as:

- CPU utilization
- Memory utilization
- Request rate, with an appropriate custom or external metrics setup
- Queue length, with an appropriate external metrics integration

HPA makes scaling decisions based on the **metric configured for it**, not directly on the number of users or incoming requests.

2.1 Understanding CPU requests and limits

Before configuring a CPU-based HPA, the application containers need CPU requests.

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

- `250m` **CPU** means 0.25 CPU core.

- `500m` **CPU** means 0.5 CPU core.
- **Requests** are used by the scheduler when placing Pods on nodes.
- **Limits** set an upper bound on resource consumption.

For a CPU-utilization-based HPA, utilization is calculated relative to the **CPU request**, not the CPU limit or total node capacity.

Example:

- Requested CPU = `250m`
- Current CPU consumption = `200m`

```
CPU utilization = (CPU usage / CPU request) × 100
                = (200m / 250m) × 100
                = 80%
```

This Pod is at **80% CPU utilization relative to its request**, even though it is below its `500m` CPU limit.

2.2 How HPA calculates the desired replica count

Assume the following configuration:

Setting	Value
Minimum replicas	2
Maximum replicas	8
Target average CPU utilization	50%
Current replicas	2
Current average CPU utilization	80%

A simplified HPA calculation is:

```
Desired replicas = ceil(Current replicas × Current utilization / Target utilization)
                 = ceil(2 × 80 / 50)
                 = ceil(3.2)
                 = 4
```

HPA may therefore increase the Deployment from **2 Pods to 4 Pods**.

> **Important:** This is a simplified calculation. Actual HPA behavior also takes into account missing metrics, Pods that are not yet ready, tolerance, and scaling stabilization rules.

2.3 Where HPA gets its metrics

HPA does not ask the Spring Boot application directly how much CPU it is using. In a typical CPU-based setup, **Metrics Server supplies resource metrics**, and the **HPA controller decides the desired replica count**.

```
Pod resource metrics
       ↓
Metrics Server
       ↓
HPA controller evaluates the configured target
       ↓
Desired replica count changes
       ↓
Deployment / ReplicaSet controllers ensure the Pods exist
```

Metrics Server supplies metrics; it does not make scaling decisions.

2.4 HPA manifest for `notes-app`

Make sure the `notes-app` Deployment has CPU requests configured, then create a file named `hpa.yaml`:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: notes-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: notes-app
  minReplicas: 2
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
```

```
        target:
          type: Utilization
          averageUtilization: 50
```

Field	Purpose
`scaleTargetRef`	The workload HPA controls (`notes-app` Deployment)
`minReplicas: 2`	Keeps at least two desired replicas
`maxReplicas: 8`	Caps the desired replicas at eight
`averageUtilization: 50`	Sets target average CPU utilization to 50% of CPU requests

The maximum replica count is a **scaling boundary**, not a guarantee that the cluster has resources to run all eight Pods.

2.5 HPA commands in Minikube

Enable Metrics Server:

```
minikube addons enable metrics-server
```

Verify that the relevant Pods are running:

```
kubectl get pods -n kube-system
```

Check resource metrics:

```
kubectl top pods
kubectl top nodes
```

Apply the HPA configuration:

```
kubectl apply -f hpa.yaml
```

Inspect HPA:

```
kubectl get hpa
kubectl describe hpa notes-app-hpa
```

Watch Pods as the replica count changes:

```
kubectl get pods -w
```

> **Practical note:** Installing Metrics Server does not create CPU load. To see a CPU-based HPA scale, the application must actually consume enough CPU to cross the configured target.

2.6 Limitation of CPU-based HPA

High traffic does not always mean high CPU usage. A Spring Boot application might spend a lot of time waiting for MySQL queries or responses from another API. In that situation:

- Users may experience long response times.
- Request queues may grow.
- CPU utilization may still stay relatively low.

A CPU-based HPA might **not** scale because the CPU metric has not crossed its target. For such workloads, production teams may use request-rate, concurrency, or queue-length metrics instead.

---

3. Vertical Pod Autoscaler (VPA)

Vertical Pod Autoscaler (VPA) helps determine more appropriate CPU and memory requests for workloads based on resource usage.

For example, a Spring Boot application might initially request:

```
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
```

After observing the application, VPA might recommend:

Resource	Previous request	Example VPA recommendation
CPU	`100m`	`400m`
Memory	`128Mi`	`512Mi`

This can help avoid **under-provisioning**, where the application has too few requested resources, or **over-provisioning**, where resources are requested far beyond typical needs.

Depending on its configuration and cluster support, VPA can either **recommend** new values or **apply** them.

3.1 Main VPA components

Component	Responsibility
Recommender	Studies resource usage and calculates CPU/memory recommendations
Updater	Determines when existing Pods should be updated
Admission Controller	Applies recommended resource values to newly created Pods

Simplified process:

```
Pod CPU / memory usage
          ↓
VPA Recommender
          ↓
Recommended CPU / memory requests
          ↓
VPA update mechanism (if enabled)
          ↓
Pod resource allocation adjusted
```

3.2 Does VPA recreate Pods?

VPA has historically applied resource changes by **evicting existing Pods**, allowing their controllers to create replacements with updated requests.

On supported Kubernetes and VPA versions, it can also make use of **in-place resource resizing**. The `InPlaceOrRecreate` mode attempts to resize a Pod in place and can fall back to recreation when needed.

For a recommendation-only setup, VPA provides an `Off` **mode**, which does not apply its recommendations automatically.

3.3 Example VPA manifest

The following example assumes that VPA components and their Custom Resource Definition (CRD) have already been installed in the cluster.

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: notes-app-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: notes-app
  updatePolicy:
    updateMode: "Off"
```

With `updateMode: "Off"`, VPA **only provides recommendations**. It does not automatically change the resources of running Pods.

> **Installation note:** VPA is an additional Kubernetes component. It is not guaranteed to be installed in a default Kubernetes or Minikube setup.

3.4 Using HPA and VPA together

HPA and VPA can be combined, but their metrics and responsibilities must be coordinated carefully.

For example:

1. An HPA targets **50% CPU utilization relative to CPU requests**.
2. VPA increases the CPU request for the same workload.
3. The calculated CPU utilization percentage changes, even if actual usage stays similar.

4. The HPA's scaling decisions can be affected.

For a typical stateless web service, a useful starting approach is to use **HPA to adjust replica count** and **VPA recommendations to right-size CPU and memory requests**. Avoid casually configuring both to compete over the same CPU or memory utilization signal.

---

4. Cluster Autoscaler

HPA can request more replicas, but those Pods still need nodes with enough available capacity.

Suppose HPA increases the desired replica count to eight. The cluster has only two worker nodes, and both are almost full. Kubernetes creates the extra Pods, but the scheduler cannot find a suitable node for some of them. Those Pods remain in the `Pending` state.

HPA does not create new worker machines. **Cluster Autoscaler** handles a different layer of capacity: the number of worker nodes in supported and configured node groups.

4.1 How Cluster Autoscaler works

1. HPA increases the desired number of application Pods.
2. The Deployment/ReplicaSet creates the additional Pods.
3. Some Pods cannot be scheduled due to insufficient node capacity.
4. Cluster Autoscaler checks whether scaling a configured node group would help those Pods run.
5. The infrastructure provider provisions additional worker nodes.
6. New nodes join the Kubernetes cluster.
7. The scheduler places the Pending Pods on nodes with suitable capacity.

```
Application demand increases
            ↓
HPA increases desired Pods
            ↓
Scheduler cannot place some Pods
```

```
            ↓
Pods remain Pending
            ↓
Cluster Autoscaler adds eligible node capacity
            ↓
Scheduler places Pending Pods
```

4.2 Cluster Autoscaler is not simply a CPU threshold

Cluster Autoscaler does **not** automatically add a node every time node CPU usage crosses 80%. It primarily responds to **scheduling requirements**, especially Pods that cannot fit on existing nodes.

The scheduler considers resource **requests** when deciding placement.

Example: A node has four allocatable CPU cores, and existing Pods have already requested all four. Even if actual CPU use is low, another Pod that requests CPU may not fit on that node.

The reverse can also happen: node CPU usage may be high, but if no additional Pods need scheduling, that alone does not mean Cluster Autoscaler will add a node.

4.3 Cluster Autoscaler in Minikube

Minikube is mainly a local development environment. Enabling HPA does **not** mean Minikube will automatically create an AWS EC2 instance or provision new worker nodes through Cluster Autoscaler.

Node autoscaling in production requires a compatible infrastructure provider and a properly configured autoscaling setup.

4.4 Scale-down behavior

Cluster Autoscaler can also scale down when nodes are no longer needed, subject to its settings and safety checks. Before removing a node, it considers whether its Pods can be moved and whether workload disruption constraints allow it.

A node is not terminated just because it currently has low CPU usage.

4.5 HPA vs. VPA vs. Cluster Autoscaler

Mechanism	What it changes	Main problem it solves
HPA	Number of workload replicas	The application needs more/fewer Pods
VPA	CPU and memory requests for Pods	Individual Pods are under- or over-provisioned
Cluster Autoscaler	Number of worker nodes	The cluster needs more/fewer machines to schedule workloads

How they fit together: HPA can ask for more Pods. If there isn't enough node capacity, Cluster Autoscaler may add nodes. VPA helps keep the resource requests of individual Pods appropriately sized.

Part 2 — RBAC, ServiceAccounts and Least Privilege

5. Why Access Control Matters in Kubernetes

A production Kubernetes cluster may run several applications and tools: Spring Boot services, MySQL databases, payment systems, analytics workloads, and monitoring components. Developers, CI/CD systems, and running applications may all interact with the cluster.

These identities should not automatically have the same permissions.

- A developer who only needs to inspect logs should not automatically be able to delete the payment Deployment.
- An application that needs to read ConfigMaps should not automatically be allowed to read Secrets or delete Pods.
- A monitoring system may need read-only access to specific Kubernetes resources, not administrative access to everything.

Kubernetes uses **Role-Based Access Control (RBAC)** to define which authenticated identities can perform particular operations on Kubernetes API resources.

---

6. Authentication vs. Authorization

Concept	Question it answers	Meaning
Authentication	Who are you?	Establishes the identity making the request
Authorization	What are you allowed to do?	Determines whether the identity has permission for the requested action

For example:

```
kubectl delete pod mysql-xyz
```

This command sends a request to the **Kubernetes API Server**. The API Server establishes the requester's identity and evaluates whether that identity is authorized to delete the Pod.

```
kubectl / client request
          ↓
Kubernetes API Server
          ↓
Authentication: identify requester
          ↓
Authorization: check requested permission
          ↓
Allow or deny the Kubernetes API operation
```

RBAC is a Kubernetes API authorization mechanism. It does not decide whether a customer is allowed to call an endpoint in your Spring Boot application. Application-level rules, such as those in Spring Security, are separate.

---

7. The Four Main RBAC Objects

Object	Purpose	Scope
Role	Defines permissions on resources within a namespace	Namespace
RoleBinding	Grants a Role's permissions (or a ClusterRole's permissions) in a namespace	Namespace

Object	Purpose	Scope
ClusterRole	Defines permissions that can include cluster-scoped resources or be reused in namespaces	Cluster-defined
ClusterRoleBinding	Grants a ClusterRole's permissions across the cluster	Cluster

A **Role or ClusterRole defines permissions**. A **RoleBinding or ClusterRoleBinding assigns those permissions to an identity**.

7.1 Role: defining permissions

The following Role allows `get` and `list` operations on ConfigMaps in the `devops-demo` namespace:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: configmap-reader
  namespace: devops-demo
rules:
  - apiGroups: [""]
    resources: ["configmaps"]
    verbs: ["get", "list"]
```

Field explanations:

- `apiGroups: [""]` — ConfigMaps belong to Kubernetes' core API group.
- `resources: ["configmaps"]` — Permissions apply to ConfigMaps.
- `verbs: ["get", "list"]` — Allows reading an individual ConfigMap and listing ConfigMaps.
- `namespace: devops-demo` — Limits the Role to that namespace.

This Role does **not** grant `create`, `update`, or `delete` operations. It also does not grant permission to read Secrets, modify Deployments, or delete Pods.

7.2 RoleBinding: assigning permissions

Defining a Role does not automatically assign it to a user or workload. A **RoleBinding** connects the permission definition to an identity.

The example below assigns the `configmap-reader` Role to the `notes-reader` ServiceAccount:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: notes-configmap-binding
  namespace: devops-demo
subjects:
  - kind: ServiceAccount
    name: notes-reader
    namespace: devops-demo
roleRef:
  kind: Role
  name: configmap-reader
  apiGroup: rbac.authorization.k8s.io
```

- `subjects` identifies who receives the permissions: `notes-reader`.
- `roleRef` identifies which permission definition is being granted: `configmap-reader`.

The permissions granted by this binding apply in the `devops-demo` **namespace**.

7.3 ClusterRole: permissions for cluster-scoped resources

Some resources, such as **Nodes**, are not part of an individual namespace.

```
kubectl get nodes
```

A monitoring application that needs to list nodes may use a ClusterRole like this:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
  - apiGroups: [""]
    resources: ["nodes"]
    verbs: ["get", "list"]
```

A **ClusterRoleBinding** can then grant this ClusterRole to the appropriate monitoring identity across the cluster.

---

8. ServiceAccounts: Identities for Workloads

Kubernetes API clients generally fall into two broad groups:

- **Human identities:** for example, a DevOps engineer running `kubectl`.
- **Workload identities:** for example, an application inside a Pod that needs to query the Kubernetes API.

A **ServiceAccount** provides a Kubernetes identity for a workload.

8.1 Creating a ServiceAccount

Create a ServiceAccount named `notes-reader` in `devops-demo`:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: notes-reader
  namespace: devops-demo
```

8.2 Assigning a ServiceAccount to a Pod

Specify `serviceAccountName` in the Pod template of the Deployment:

```
spec:
  template:
    spec:
      serviceAccountName: notes-reader
      containers:
        - name: notes-app
          image: your-image:latest
```

The image is a placeholder; use the actual application image in your Deployment. The Deployment/Pods and ServiceAccount must be in the intended namespace.

Now the Pod can use the `notes-reader` identity when it authenticates to the Kubernetes API.

8.3 How ServiceAccount authentication works

In a typical modern cluster:

1. A Pod is assigned a ServiceAccount.
2. Kubernetes can project a short-lived ServiceAccount token into the Pod, unless automatic token mounting is disabled.
3. Application code or a Kubernetes client library uses that token to authenticate to the API Server.
4. The API Server checks which operations the ServiceAccount is allowed to perform.

```
Application in Pod
       ↓
Projected ServiceAccount token
       ↓
Kubernetes API Server authenticates identity
       ↓
RBAC rules determine authorized actions
```

A ServiceAccount provides an **identity**; it does not, by itself, grant broad access. The relevant RoleBindings or ClusterRoleBindings determine what the identity can do.

---

9. Does Every Spring Boot App Need a Custom ServiceAccount?

No. A typical Spring Boot application may only handle HTTP requests and connect to MySQL:

```
HTTP request → Spring Boot controller → MySQL
```

If it never calls the Kubernetes API, it does not need Kubernetes API permissions to list Pods, read ConfigMaps, or update Deployments.

By default, a Pod uses its namespace's `default` **ServiceAccount** if no other ServiceAccount is specified. However, there is no need to give that identity

additional permissions without a valid reason.

Disabling automatic ServiceAccount token mounting

For an application that does not need Kubernetes API access, automatic token mounting can be disabled in the Pod template:

```
spec:
  template:
    spec:
      automountServiceAccountToken: false
```

This helps prevent a Kubernetes API credential from being exposed if the container is compromised.

> **Keep these separate:** ServiceAccount tokens authenticate workloads to the **Kubernetes API**. They do not allow a Spring Boot application to connect to MySQL. Database connections use ordinary networking and database-specific credentials.

---

10. Least Privilege in Production

The **principle of least privilege** means giving an identity only the permissions needed to perform its job.

Consider a compromised Spring Boot container:

Configuration	Potential impact
Excessive permissions: workload has `cluster-admin`	An attacker with access to its usable Kubernetes credentials may be able to read or modify extensive cluster resources.
Least privilege: workload can only read ConfigMaps in its namespace	Access through that Kubernetes identity is much more restricted; it cannot use that Role to delete Deployments, read Secrets, or modify cluster-wide resources.

The goal is to **limit what an application or user can do if their credentials are misused**.

Least privilege applies to developers, CI/CD pipelines, monitoring tools, Kubernetes controllers, and application Pods.

Quick Revision

Question	Answer
What scales the number of Pods?	HPA
What recommends or adjusts Pod CPU/memory requests?	VPA
What adjusts the number of worker nodes?	Cluster Autoscaler
Does a CPU-based HPA measure utilization against CPU limits?	No. It measures against CPU **requests**.
Does HPA automatically provision nodes?	No.
Can Cluster Autoscaler add nodes solely because CPU usage is high?	Not normally; it primarily responds to scheduling needs.
What does a Role define?	Permissions for resources within a namespace.
What does a RoleBinding do?	Grants defined permissions to an identity in a namespace.
What is a ServiceAccount?	A workload identity for Kubernetes API authentication.
Does every Spring Boot app need Kubernetes API permissions?	No.
What does least privilege mean?	Only grant the permissions required for the job.